

Backend Developer — Technical Assessment

Time allocation: approximately 1–2 evenings **Technology:** Java with Spring Boot **Submission:** a link to a Git repository

Instructions

Read this document in full before beginning. Read Part 2 before starting Part 1, as it affects the design decisions you make in Part 1. This assessment evaluates engineering quality — structure, correctness, testing, and reasoning — not the volume of features delivered. A small, well-structured, well-tested service will score higher than a large, disorganised one.

Part 1 — Implement the service

Implement an **Order Service**: a backend for a simple e-commerce shop. A customer places an order consisting of one or more line items; the order is then read, updated, and managed through its lifecycle.

Data model

Implement the **Order** entity with the following six fields:

Field	Type	Constraints
orderId	UUID	Server-generated. Must not be accepted or overridden by the client.
customerName	string	Required.
items	list	At least one line item. An order with no items is invalid.
status	enum	Lifecycle: CREATED → PAID → SHIPPED → DELIVERED , plus CANCELLED . Defaults to CREATED .
totalAmount	decimal	The order total. Decide whether this is client-supplied or server-computed.
createdAt	timestamps	Server-managed.
updatedAt	timestamps	Server-managed.

Each line item contains a **productName**, a **quantity**, and a **unitPrice**.

Any detail not specified above (rounding, currency, validation limits, additional fields) is left to your discretion. State the assumptions you make.

Requirements (mandatory)

The following four requirements are mandatory. All four must be satisfied for a complete submission.

1. **Full CRUD.** Implement create, read (single and list), update, and delete operations. Data must persist across requests.
2. **RESTful API design.** The API must follow REST conventions: resource-oriented URLs, correct HTTP methods, appropriate status codes for both success and failure, consistent request and response payloads, input validation, and correct handling of invalid input and missing resources. Define the error response format yourself.
3. **Current Spring Boot.** Use the latest stable release of Spring Boot and a currently supported version of Java. State both versions in the README.
4. **Unit tests.** Provide unit tests covering the business logic, including failure paths — not only the success path. Tests must be runnable with a single command.

Reference requests

The exact URLs, payload shapes, field names, and error responses are your design. The following illustrate the domain only.

Create an order

```
curl -X POST http://localhost:8080/api/orders \
-H "Content-Type: application/json" \
-d '{
    "customerName": "Andi Wijaya",
    "items": [
        { "productName": "Apple",      "quantity": 3, "unitPrice": 0.50
    },
        { "productName": "Bread Loaf", "quantity": 1, "unitPrice": 2.20 }
    ]
}'
```

Expected: the order is created and returned with a server-assigned **orderId**, **status** of **CREATED**, and a computed total of **3.70**.

Read

```
curl http://localhost:8080/api/orders/{id}           # → the order
curl http://localhost:8080/api/orders/{unknown-id}  # → an appropriate
not-found response
```

List — paginated and orderable (see Part 2):

```
curl "http://localhost:8080/api/orders?page=0&size=20&sort=newest"
```

Update, cancel, and delete — the corresponding operations, behaving correctly.

Account for cases not shown above: an empty item list, a negative quantity, a total that does not match the line items, and an unknown order id. Defining the correct behaviour for these cases is part of the assessment.

Part 2 — Extend the service function

The following requirements are introduced after Part 1. Implement them. The amount of existing code you must modify to accommodate them is itself part of the evaluation.

- **Status transitions are constrained.** An order cannot become **SHIPPED** before it is **PAID**, and a **DELIVERED** or **CANCELLED** order cannot be reactivated or shipped again. Reject illegal transitions cleanly; define the response returned in that case. Example: a request to ship an order still in **CREATED** must be refused.
 - **Certain transitions require additional data.** Cancelling an order requires a reason; marking it paid or shipped does not. Assume such rules will continue to be added.
 - **Items are immutable after payment.** Once an order is **PAID**, its line items may no longer be modified, even if other fields remain editable.
 - **The list endpoint must support multiple ordering rules** — for example: newest first, highest total value first, or oldest unpaid orders first. Assume new ordering rules will continue to be added, and that adding one must not require rewriting the existing rules.
-

Optional Assessments

Attempt these only after the mandatory requirements are complete. Depth is preferred over breadth: one extension implemented thoroughly is worth more than several implemented superficially. Select one or two, and justify your approach in the README.

- **Security.** Address access control, handling of malformed or hostile input (including requests attempting to set **orderId**, **status**, or **totalAmount** directly), and information disclosure in responses.
 - **Deployment.** Make the service straightforward to build and run in a container environment.
 - **Robustness.** Be explicit about the inputs and states the service must reject (negative quantities, totals inconsistent with line items, illegal transitions), and demonstrate this with tests.
 - **Architecture.** Structure the codebase so that it is navigable and extensible — for example, adding a new order status or ordering rule with minimal disruption. If you apply recognised design patterns, name them and explain the choice.
-

Deliverables

1. A **Git repository** with incremental, well-described commits. Committing Part 1 and Part 2 separately is recommended.
2. A **README** containing:
 - The exact Spring Boot and Java versions used.
 - Instructions to build, run, and test the service.

- Instructions to exercise the API (curl commands, a Postman/Bruno collection, or Swagger).
- A **Design Decisions** section stating your assumptions, any scope deliberately omitted, how the Part 2 requirements affected your Part 1 design, and what you would improve given more time. This section is assessed.

3. A passing test suite.

Conditions

- Complete the mandatory requirements before attempting the optional extensions.
- The use of AI assistants is permitted. You must fully understand your submission and be able to explain any part of it on request; a code walkthrough may be conducted.
- Where the specification is ambiguous, make a reasonable decision and document it.
- Do not add functionality beyond what is specified at the expense of the specified requirements.